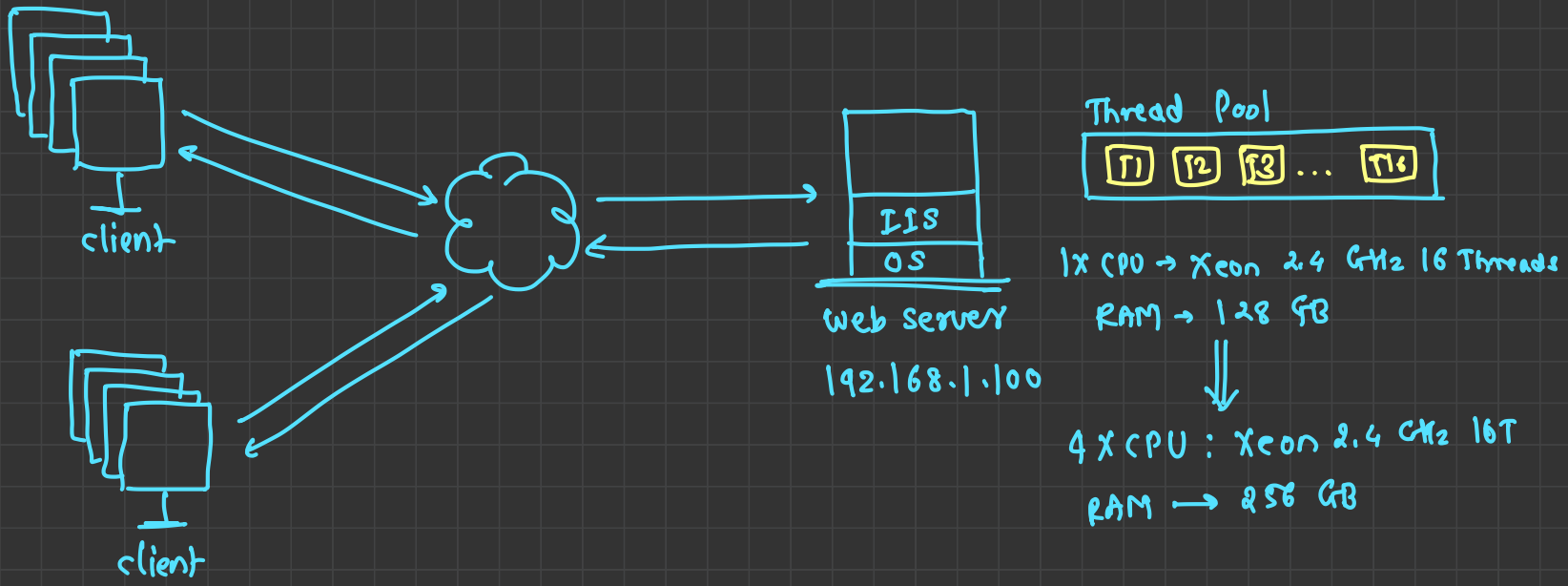
Scalability


client
client
IIS
OS
Web server
192.168.1.100
Thread Pool
T1
T2
T3
...
T16
1X CPU → Xeon 2.4 GHz 16 Threads
RAM → 128 GB
4 X CPU : Xeon 2.4 GHz 16T
RAM → 256 GB

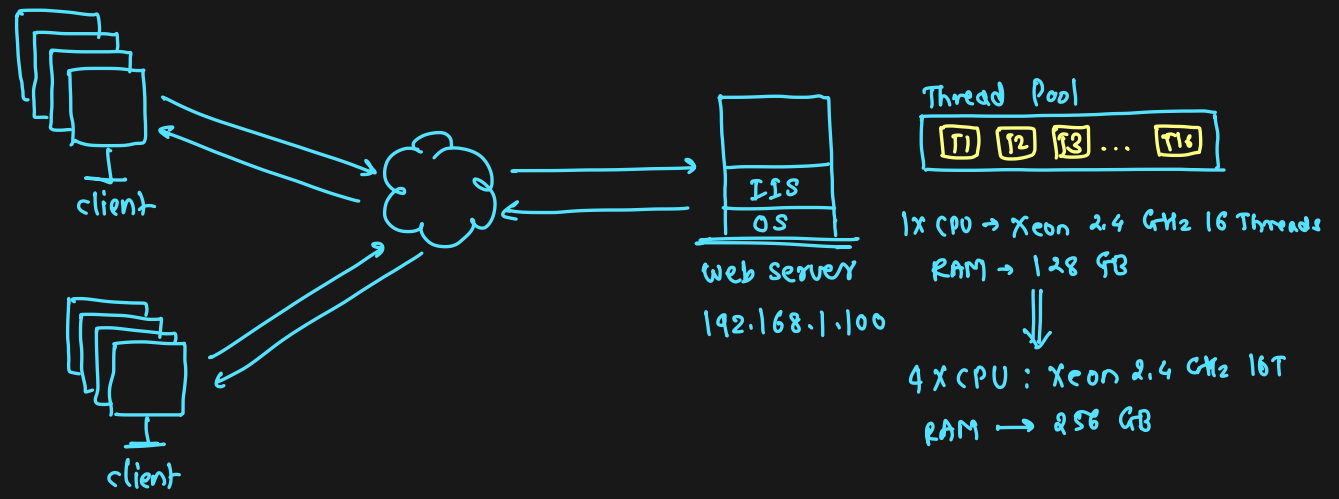
Scalability

- The ability of a computer application or product (hardware or software) to continue to function well when it (or its context) is changed in size or volume in order to meet a user need. Typically, the rescaling is to a larger size or volume
- The rescaling can be of the product itself (for example, a line of computer systems of different sizes in terms of storage, RAM, and so forth) or in the scalable object's movement to a new context (for example, a new operating system)
- It is the ability not only to function well in the rescaled situation, but to actually take full advantage of it
- For example, an application program would be scalable if it could be moved from a smaller to a larger operating system and take full advantage of the larger operating system in terms of performance (user response time and so forth) and the larger number of users that could be handled

types
- vertical
- Horizontal

Vertical Scaling

- Vertical scaling (or "scaling up") refers to adding more hardware to an existing machine so that you run the same workload on better specs
- For example, if a server requires more processing power, vertically scaling the device would mean upgrading its CPU. Other examples of vertical scaling include:
  - Adding more RAM to a machine
  - Upgrading a hard disk drive to an SSD with more storage
  - Improving network speed
- This type of scaling was the go-to option back when we had monolithic apps that had to run on a single server
- Nowadays, vertical scaling is a common choice for small-to-mid-size companies that see a noticeable difference in performance or UX from an upgraded machine



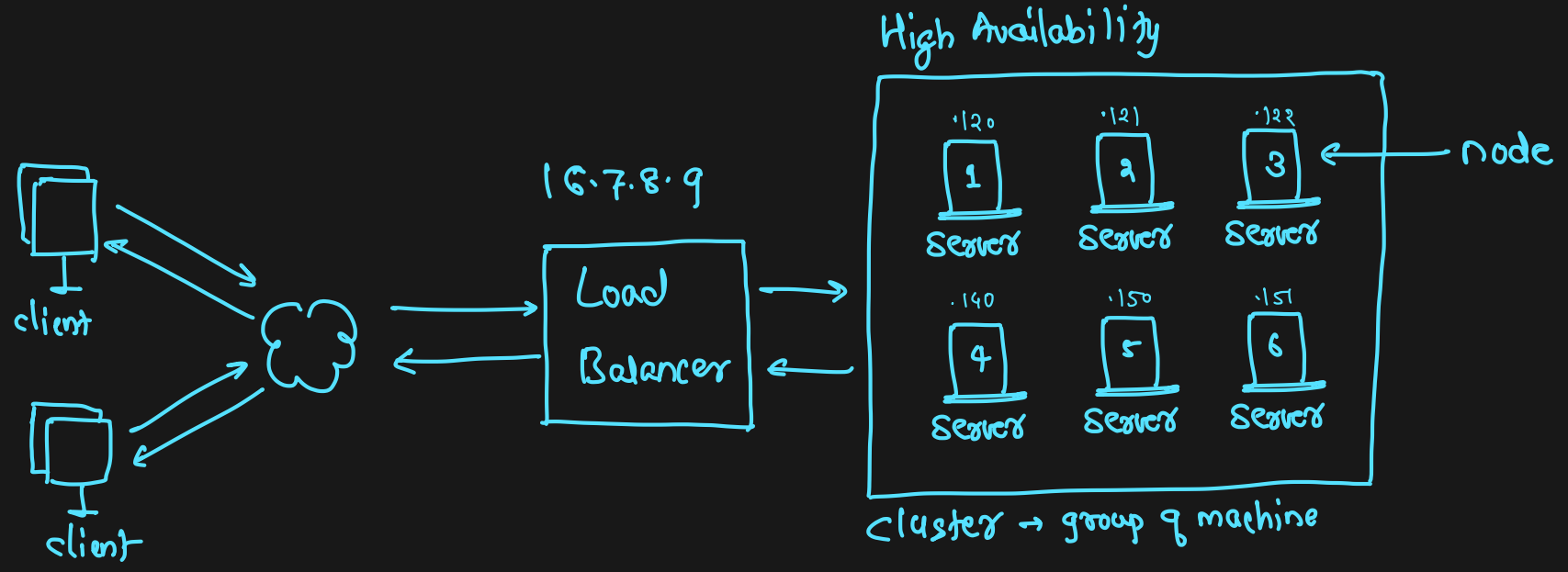
Vertical Scaling



- Advantages
  - Fast response times as there are no synchronized nodes that communicate with each other
  - Upgrading specific components costs less than purchasing a new server
  - You need less space in a server room to scale vertically than horizontally (cooling and power bills are also lower).
  - Everything runs on a single node, which leads to better data consistency and integrity
  - Easy to implement as there is no need for load balancing or virtualization
  - You do not add any extra complexity to day-to-day IT management
- Disadvantages
  - There's a limit to how much a single machine can scale since each device has a threshold for RAM, storage, and processing power
  - Higher possibility for service downtime as you rely on one machine that acts as a single point of failure
  - You must take the machine offline to perform the upgrade → downtime
  - There is always a percentage of computing resources you are not using actively
  - Increased risk of permanent data loss due to hosting everything on a single servers

Horizontal Scaling

- Horizontal scaling (or "scaling out") refers to adding more nodes to a system's resource pool to meet the increased demand.
- You do not improve the specifications of the existing machine—instead, you add more same-size servers to the cluster and share the workload across more devices
- Scaling out enables a sysadmin to combine the power of multiple machines to meet the current load demands
- Nodes do not have to be in the same data center, they do not have to be in the same region even



Horizontal Scaling



- Advantages
  - Systems that scale out have no scalability cap. The only limits to how much you scale horizontally are the budget and data center space
  - Distributing data and operations among nodes increases fault tolerance. If one of the servers goes down, the rest of the system keeps services online and maintains availability
  - The system does not go offline during the scaling process
  - Systems that scale out also enable you to reduce capacity seamlessly, so you're not stuck always paying for peak demand
  - A skilled team knows how to set up a system that scales out automatically (in both ways)
  - Sysadmins get to distribute data backups across several nodes
- Disadvantages
  - You must add software for load balancing and ensure nodes synchronize effectively. This setup is challenging for less experienced teams
  - Day-to-day server management gets more complex the more nodes you add to a system
  - Data consistency is a concern due to cross-server communication
  - Node-to-node communication slows down performance in a poorly optimized cluster
  - Servers still encounter hardware limits if machines are too small
  - Each new server requires more space, cooling, and power, so the costs of scaling out quickly spiral out of control
  - More nodes create a larger attack surface and lead to more complex incident response planning

Load Balancer

Load Balancer

cluster of nodes

- A **load balancer** is a system (hardware, software, or cloud service) that sits between **clients** and a **pool of servers** and distributes incoming network traffic across those servers
- Think of it as a **traffic cop** at a busy intersection
- Instead of every car (client request) piling onto one road (server), the officer directs each car to a different lane (server) so no single road gets overwhelmed

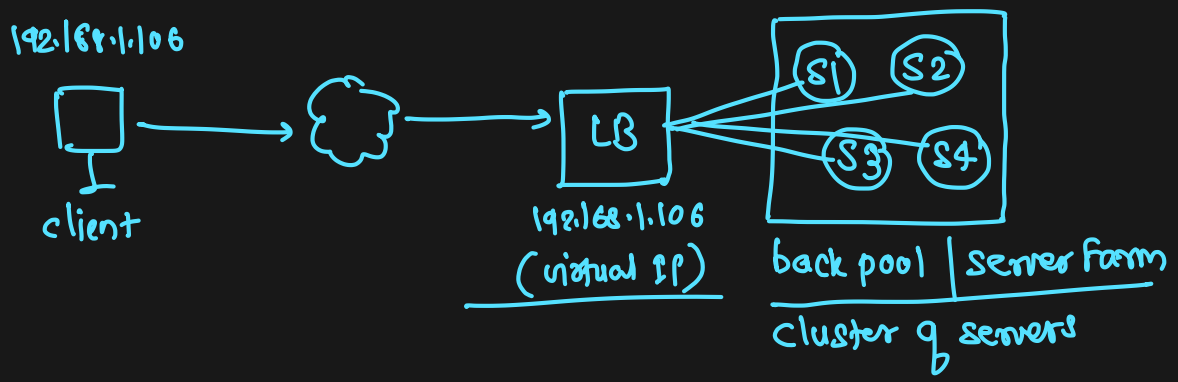
  - load balancer checks health of all the nodes by sending a beat beat signal
    - ↳ network packet

Why Load Balancer

- **Scalability (Handling More Traffic)**
  - A single server has finite CPU, RAM, and network capacity
  - When traffic grows, one machine cannot keep up
  - A load balancer lets you add more servers (**scale out**) instead of buying a bigger one (**scale up**), which is cheaper and practically unlimited → horizontal scaling
- **High Availability (Avoiding Downtime)**
  - If one server crashes, the load balancer detects the failure and stops sending traffic to it
  - Users experience no interruption because the remaining servers absorb the load. This eliminates the **single point of failure**
- **Performance & Efficiency**
  - By spreading requests evenly, no server is overworked while others sit idle
  - Response times stay consistent, and hardware is used efficiently

Components of Load Balancer

- **Virtual IP (VIP) / Cluster IP:** The single public-facing IP address clients connect to. Owned by the load balancer, not any individual server
- **Backend Pool / Server Farm:** The group of real servers that actually process the requests
- **Health Probes / Heartbeats:** Periodic checks the load balancer performs to verify each server is alive and responding
- **Load Balancing Algorithm:** The rule that decides which server gets the next request
- **Listener / Port Rules:** Defines which ports and protocols (TCP/UDP) the load balancer accepts and how to handle them
- **Session Persistence (Affinity):** Ensures a client keeps talking to the same server when needed → stateful

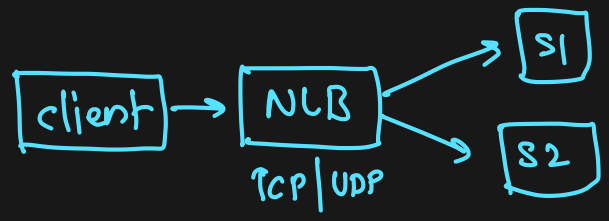


How a Load Balancer Works

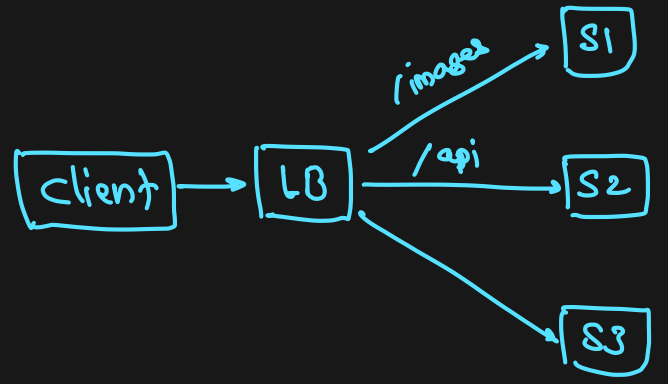
- **Client sends a request** to the VIP (e.g., 192.168.1.100:80)
- **The load balancer receives it** and consults its algorithm
- **It picks a backend server** based on health, load, and affinity rules
- **It forwards the request** to that server (or rewrites the destination)
- **The server processes the request** and returns a response
- **The load balancer relays the response** back to the client
- **Health probes run continuously** so failed servers are removed from rotation

Types of Load Balancers

- **Layer 4 (L4) — Transport Layer**
  - Works with **TCP and UDP** — IP addresses and ports only
  - Does **not** inspect the content of packets (no URLs, no cookies)
  - **Extremely fast** because it just forwards packets
  - Cannot make smart decisions based on application data
  - **Examples:** Windows NLB, AWS NLB, F5 LTM (in L4 mode), HAProxy (TCP mode)



- **Layer 7 (L7) — Application Layer**
  - Works with **HTTP, HTTPS, FTP, DNS**, etc
  - Can **inspect content**: URL paths, headers, cookies, hostnames
  - Can route /images to one server and /api to another
  - Can terminate SSL, rewrite URLs, compress responses
  - **Slower** than L4 due to deeper inspection, but far smarter
  - **Examples:** IIS ARR, NGINX, HAProxy (HTTP mode), AWS ALB, Azure App Gateway



Load Balancing Algorithms

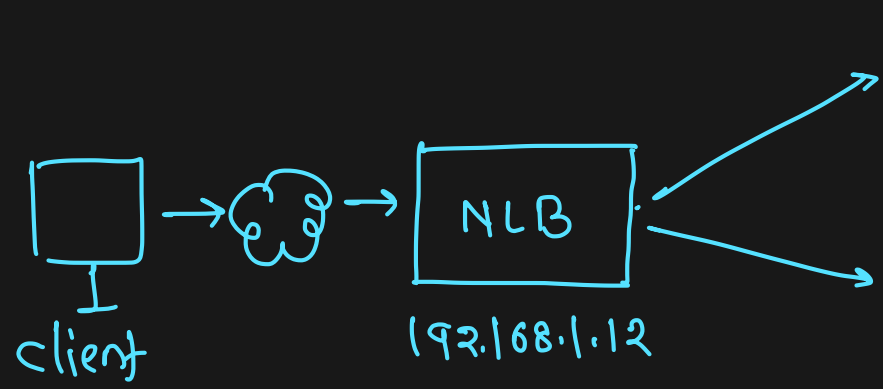


- Round Robin
  - Sends requests to servers in sequential order (1→2→3→1→2→3)
  - Servers with equal capacity; stateless apps
- Least Connections
  - Sends to the server with the fewest active connections
  - Apps with long-lived sessions
- Least Response Time
  - Sends to the fastest-responding server
  - Latency-sensitive apps
- IP Hash
  - Hashes client IP to always pick the same server
  - Session persistence without cookies
- Weighted
  - Servers with more capacity get more requests
  - Mixed hardware in a pool
- Random
  - Picks any server randomly
  - Simple, uniform environments

Session Persistence



- Some applications store user state in memory on the server (e.g., a shopping cart).
- If the next request goes to a different server, the session is lost
- Affinity solves this by ensuring a client always reaches the same server
  - None: Any server can handle any request. Best for stateless apps.
  - Single (Source IP): A client IP always maps to the same server.
  - Network (Class C): A client's entire /24 subnet maps to the same server.
- Trade-off: Affinity reduces even distribution and can create hotspots if one client sends heavy traffic.



Network Load Balancer

- Network Load Balancing (NLB) is a Windows Server software feature that distributes incoming TCP/IP traffic across multiple servers (a cluster) to ensure high availability and scalability
- From the client's perspective, the cluster appears as a single virtual server with one IP address
- NLB operates at **Layer 4 (Transport Layer)** of the OSI model, balancing TCP and UDP traffic
- NLB uses a distributed filtering algorithm on each host, meaning **no central dispatcher** exists
- All hosts in the cluster simultaneously receive incoming packets and independently decide whether to accept or discard them based on the client's IP and port mapping

Features & Concepts



- **Cluster IP (Virtual IP)**
  - The shared IP address that clients use to connect. It is assigned to all hosts in the cluster
- **Convergence**
  - When a host fails or is added, the remaining hosts exchange heartbeat messages (typically within 10 seconds) to determine a new load distribution
  - Traffic is then redistributed
- **Port Rules**
  - Define which TCP/UDP ports are load-balanced, blocked, or handled by a single host (default host)
  - This allows fine-grained control over specific services like HTTP (port 80) or HTTPS (443)
- **Affinity (Session Persistence)**
  - Controls whether a client's requests are always directed to the same host
    - None: No affinity; any host can respond
    - Single: A client IP always maps to the same host
    - Network: A client's /24 subnet maps to the same host
- **Operation Modes**
  - Unicast: All hosts share one MAC address. Simple but can cause network switch flooding
  - Multicast: Each host keeps its unique MAC but also accepts the cluster's multicast MAC. Requires proper switch configuration but scales better

Configuration

- **Step 1: Install the NLB Feature**
  - Open **Server Manager**.
  - Click **Add roles and features**.
  - Click **Next** through the wizard until you reach **Features**.
  - Check the box for **Network Load Balancing**.
  - Click **Install** (no reboot required).
- **Step 2: Create the NLB Cluster**
  - In Server Manager, click **Tools > Network Load Balancing Manager**.
  - Right-click **Network Load Balancing Clusters** and select **New Cluster**.
  - **Host:** Type the name or IP of the **first** server. Click **Connect**.
  - Select the network interface that will handle cluster traffic (e.g., "Ethernet"). Click **Next**.
  - **Host Parameters:** Leave the **Priority** as 1 for the first host. Click **Next**.
  - **Cluster IP Addresses:** Click **Add**. Enter your virtual Cluster IP (e.g., 192.168.1.100) and Subnet Mask. Click OK, then Next.
  - **Cluster Parameters:**
    - **Full Internet Name:** Enter a name (e.g., webfarm.contoso.com).
    - **Operation Mode:** Select **Multicast** (recommended for better switch compatibility) or Unicast. Click **Next**.
  - **Port Rules:** You can use the default rule (all ports) or edit it. For a web server, click **Edit** and set the **Port Range** to 80 (and add another for 443). Set **Affinity** based on your app (usually "Single" for web sessions). Click **Finish**

Configuration

- **Step 3: Add Additional Servers**
  - In the NLB Manager, right-click the newly created cluster (e.g., webfarm.contoso.com) and select **Add Host to Cluster**.
  - Type the name/IP of the **second** server, click **Connect**, and select the interface.
  - **Host Parameters:** This host will automatically get the next available **Priority** (e.g., 2). Click **Next** and **Finish**.
  - **Verify:** The second host will appear. The status icon will turn **green** when it is successfully converged.
- **Step 4: Verification & Monitoring**
  - **Client Test**
    - Open a browser on a client machine and navigate to the **Cluster IP**.
    - You should receive responses
    - To see which server is responding, you can put a distinct identifier (like "Server 1") on each host's default web page
  - **Monitoring**
    - In the NLB Manager, you can see the status of each host (**Converged** or **Converging**)
    - You can also right-click a host to **Start/Stop** it for maintenance, which triggers convergence and redirects traffic to the remaining host